

Chapter 16. Persistent Volumes

Persistent volumes are a specific category of the wider concept of volumes with the capability of persisting data beyond a Pod lifecycle. The mechanics for persistent volumes are slightly more complex. The persistent volume is the resource that actually persists the data to an underlying physical storage.

The *persistent volume claim* represents the connecting resource between a Pod and a persistent volume responsible for requesting the storage.

Finally, the Pod needs to *claim* the persistent volume and mount it to a directory path available to the containers running inside of the Pod.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objectives:

- Implement storage classes and dynamic volume provisioning
 - Configure volume types, access modes, and reclaim policies
 - Manage persistent volumes and persistent volume claims
-

[Figure 16-1](#) shows the relationship between the Pod, the persistent volume claim, and the persistent volume.

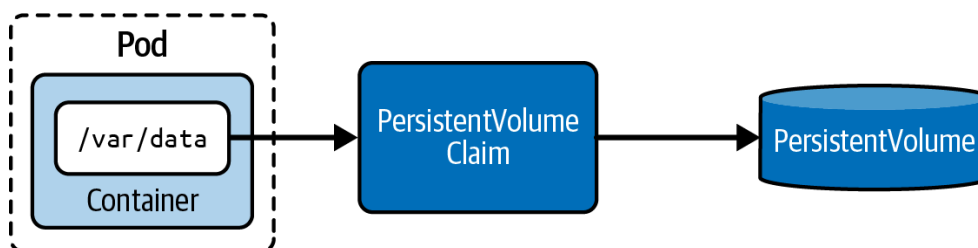


Figure 16-1. Claiming a persistent volume from a Pod

Working with Persistent Volumes

Data stored on volumes outlive a container restart. In many applications, the data lives far beyond the lifecycles of the applications, container, Pod, nodes, and even the clusters themselves. Data persistence ensures the lifecycles of the data are decoupled from the lifecycles of the cluster re-

sources. A typical example would be data persisted by a database. That's the responsibility of a persistent volume. Kubernetes models persist data with the help of two primitives: the `PersistentVolume` and the `PersistentVolumeClaim`.

The *PersistentVolume* is the primitive representing a piece of storage in a Kubernetes cluster. It is completely decoupled from the Pod and therefore has its own lifecycle. The object captures the source of the storage (e.g., storage made available by a cloud provider). A `PersistentVolume` is either provided by a Kubernetes administrator or assigned dynamically by mapping to a storage class.

The *PersistentVolumeClaim* requests the resources of a `PersistentVolume`—for example, the size of the storage and the access type. In the Pod, you will use the type `persistentVolumeClaim` to mount the abstracted `PersistentVolume` by using the `PersistentVolumeClaim`.

Volume Types

Kubernetes supports several types of persistent volumes to accommodate different storage backends and use cases. Each type has its own characteristics and is suitable for specific scenarios. [Table 16-1](#) lists the most commonly used persistent volume types that are not deprecated.

Table 16-1. Persistent volume types

Type	Description
<code>hostPath</code>	Mounts a file or directory from the host node's filesystem into the Pod. Useful for development and testing but not recommended for production multi-node clusters, as it ties the Pod to a specific node.
<code>local</code>	Represents a mounted local storage device such as a disk, partition, or directory. Provides better performance than remote storage but requires node affinity to ensure Pods are scheduled on the correct node.
<code>nfs</code>	Allows multiple Pods to share the same Network File System (NFS) mount. Supports <code>ReadWriteMany</code> access mode and is useful for sharing data between Pods across nodes.
<code>csi</code>	Container Storage Interface (CSI) driver that provides a standardized way to expose storage systems to containerized workloads. Most modern storage solutions use CSI drivers.
<code>fc</code>	Fibre Channel (FC) volume that allows existing FC storage to be attached to Pods. Requires FC hardware and proper configuration on nodes.
<code>iscsi</code>	Internet Small Computer Systems Interface (iSCSI) volume that allows existing iSCSI storage to be mounted to Pods. Provides block-level storage over IP networks.

The choice of volume type depends on your infrastructure, performance requirements, and whether you need the storage to be shared across multiple Pods or nodes. For cloud environments, CSI drivers are typically provided by cloud providers (like AWS EBS CSI driver, GCE Persistent Disk CSI driver, or Azure Disk CSI driver) to integrate with their native storage services.

Static Versus Dynamic Provisioning

A `PersistentVolume` can be created statically or dynamically. If you go with the static approach, then you first need to create a storage device and then reference it by explicitly creating an object of kind `PersistentVolume`. The dynamic approach doesn't require you to create a `PersistentVolume` object. It will be automatically created from the `PersistentVolumeClaim` by setting a storage class name using the attribute `spec.storageClassName`.

A *storage class* is an abstraction concept that defines a class of storage device (e.g., storage with slow or fast performance) used for different appli-

cation types. It's the job of a Kubernetes administrator to set up storage classes. For a deeper discussion on storage classes, see [“Storage Classes”](#). For now, we'll focus on the static provisioning of PersistentVolumes.

Creating PersistentVolumes

When you create a PersistentVolume object yourself, we refer to the approach as *static provisioning*. A PersistentVolume can be created only by using the manifest-first approach. At this time, `kubectl` doesn't allow the creation of a PersistentVolume using the `create` command. Every PersistentVolume needs to define the storage capacity using `spec.capacity` and an access mode set via `spec.accessModes`. See [“Configuration Options for a PersistentVolume”](#) for more information on the configuration options available to a PersistentVolume.

[Example 16-1](#) creates a PersistentVolume named `db-pv` with a storage capacity of 1 Gi and read/write access by a single node. The attribute `hostPath` mounts the directory `/data/db` from the host node's filesystem. We'll store the YAML manifest in the file `db-pv.yaml`.

Example 16-1. YAML manifest defining a PersistentVolume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: db-pv
spec:
  capacity: 1Gi ❶
  storage: 1Gi ❶
  accessModes: ❷
  - ReadWriteOnce ❷
  hostPath:
    path: /data/db
```

❶ The storage capacity available to the persistent volume

❷ The read/write access modes applicable to the persistent volume

When you inspect the created PersistentVolume, you'll find most of the information you provided in the manifest. The status `Available` indicates that the object is ready to be claimed. The reclaim policy determines what should happen with the PersistentVolume after it has been released from its claim. By default, the object will be retained. The following example uses the short-form command `pv` to avoid having to type `persistentvolume`:

```
$ kubectl apply -f db-pv.yaml
persistentvolume/db-pv created
$ kubectl get pv db-pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	\
CLAIM	STORAGECLASS	REASON	AGE		
db-pv	1Gi	RWO	Retain	Available	\
			10s		

Configuration Options for a PersistentVolume

A PersistentVolume offers a variety of configuration options that determine its innate runtime behavior. For the exam, it’s important to understand the volume mode, access mode, reclaim policy, and node affinity configuration options.

Volume Mode

The volume mode handles the type of device. That’s a device either meant to be consumed from the filesystem or backed by a block device. The most common case is a filesystem device. You can set the volume mode using the attribute `spec.volumeMode` . [Table 16-2](#) shows all available volume modes.

Table 16-2. PersistentVolume volume modes

Type	Description
Filesystem	Default. Mounts the volume into a directory of the consuming Pod. Creates a filesystem first if the volume is backed by a block device and the device is empty.
Block	Used for a volume as a raw block device without a filesystem on it.

The volume mode is not rendered by default in the console output of the `get pv` command. You will need to provide the `-o wide` command-line option to see the `VOLUMEMODE` column, as shown here:

```
$ kubectl get pv -o wide
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	\
CLAIM	STORAGECLASS	REASON	AGE	VOLUMEMODE	
db-pv	1Gi	RWO	Retain	Available	\
			19m	Filesystem	

Access Mode

Each PersistentVolume can express how it can be accessed using the attribute `spec.accessModes`. For example, you can define that the volume can be mounted only by a single Pod in a read or write mode or that a volume is read-only but accessible from different nodes simultaneously. [Table 16-3](#) provides an overview of the available access modes. The short-form notation of the access mode is usually rendered in outputs of specific commands, e.g., `get pv` or `describe pv`.

Table 16-3. PersistentVolume access modes

Type	Short form	Description
<code>ReadWriteOnce</code>	<code>RWO</code>	Read/write access by a single node
<code>ReadOnlyMany</code>	<code>ROX</code>	Read-only access by many nodes
<code>ReadWriteMany</code>	<code>RWX</code>	Read/write access by many nodes
<code>ReadWriteOncePod</code>	<code>RWOP</code>	Read/write access mounted by a single Pod

The following command parses the access modes from the PersistentVolume named `db-pv`. As you can see, the returned value is an array underlining the fact that you can assign multiple access modes at once:

```
$ kubectl get pv db-pv -o jsonpath='{.spec.accessModes}'  
[ "ReadWriteOnce" ]
```

`ReadWriteOnce` (`RWO`) allows mounting by a single node as read-write, ideal for databases like MySQL or PostgreSQL and stateful applications using block storage (AWS EBS, GCE Persistent Disk). `ReadOnlyMany` (`ROX`) enables multiple nodes to mount read-only simultaneously, perfect for serving static web content or shared configuration files across multiple Pods. `ReadWriteMany` (`RWX`) permits concurrent read-write access from multiple nodes, requiring file-based storage like NFS or AWS EFS, is essential for shared upload directories or content management systems where multiple Pods process the same files. `ReadWriteOncePod` (`RWOP`) ensures only one Pod cluster-wide can mount the volume, providing stronger guarantees than `RWO` for applications like etcd or during StatefulSet migrations where absolute single-writer semantics are critical. Storage provider support varies: block storage typically supports `RWO`/`RWOP` while file systems are needed for `ROX`/`RWX`.

Reclaim Policy

Optionally, you can also define a reclaim policy for a PersistentVolume. The reclaim policy specifies what should happen to a PersistentVolume object when the bound PersistentVolumeClaim is deleted (see [Table 16-4](#)). For dynamically created PersistentVolumes, the reclaim policy can be set via the attribute `.reclaimPolicy` in the storage class. For statically created PersistentVolumes, use the attribute `spec.persistentVolumeReclaimPolicy` in the PersistentVolume definition.

Table 16-4. PersistentVolume reclaim policies

Type	Description
Retain	Default. When PersistentVolumeClaim is deleted, the PersistentVolume is “released” and must be manually reclaimed.
Delete	Deletion removes PersistentVolume and its associated storage.
Recycle	This value is deprecated. You should use one of the other values.

This command retrieves the assigned reclaim policy of the PersistentVolume named `db-pv`:

```
$ kubectl get pv db-pv -o jsonpath='{.spec.persistentVolumeReclaimPolicy}'
Retain
```

Node Affinity

Node affinity allows you to constrain which nodes a PersistentVolume can be accessed from. This is particularly important for local storage types like `local` and `hostPath` volumes, which are physically tied to specific nodes. By defining node affinity rules, you ensure that Pods using the PersistentVolume are scheduled only on nodes that can actually access the underlying storage.

The node affinity is specified using the `spec.nodeAffinity` field in the PersistentVolume definition. It uses the same syntax as Pod node affinity, with `required` rules that must be satisfied for the volume to be accessible.

[Example 16-2](#) illustrates an example of a PersistentVolume with node affinity that restricts it to specific nodes.

Example 16-2. Defining node affinity for a PersistentVolume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: local-storage
  local: ❶
    path: /mnt/data
  nodeAffinity: ❷
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname ❸
              operator: In
              values:
                - node01
                - node02
```

❶ Uses the `local` volume type, which requires node affinity

❷ Defines node affinity rules for this PersistentVolume

❸ Restricts the volume to nodes with hostnames `node01` or `node02`

Common use cases for node affinity with PersistentVolumes include:

Local volumes

Must specify node affinity to indicate which node contains the storage

Zone constraints

Ensuring volumes are accessed only from nodes in specific availability zones

Hardware requirements

Restricting volumes to nodes with specific storage hardware (e.g., SSD nodes)

When a PersistentVolumeClaim binds to a PersistentVolume with node affinity, any Pod using that claim will be scheduled according to these

constraints. If no suitable node is available, the Pod will remain in Pending status.

Important considerations:

- Node affinity is required for `local` volume types.
- The scheduler considers both the Pod’s node selectors and the PersistentVolume’s node affinity.
- Changes to node labels after binding don’t affect existing mounted volumes.
- For high availability, avoid overly restrictive node affinity rules.

Creating PersistentVolumeClaims

The next object we’ll need to create is the PersistentVolumeClaim. Its purpose is to bind the PersistentVolume to the Pod. Let’s look at the YAML manifest stored in the file *db-pvc.yaml*, shown in [Example 16-3](#).

Example 16-3. Definition of a PersistentVolumeClaim

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: db-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ""
  resources:
    requests:
      storage: 256Mi
```

- ❶ The access modes we’re asking an unbound persistent volume to provide
- ❷ Uses an empty string assignment to indicate that we want to use static provisioning
- ❸ The minimum amount of storage an unbound persistent volume needs to have available

What that’s saying is: “Give me a PersistentVolume that can fulfill the resource request of 256 Mi and provides the access mode `ReadWriteOnce`.”

Static provisioning should use an empty string for the attribute `spec.storageClassName` if you do not want it to automatically assign the default storage class. The binding to an appropriate `PersistentVolume` happens automatically based on those criteria.

After creating the `PersistentVolumeClaim`, the status is set as `Bound`, which means that the binding to the `PersistentVolume` was successful. Once the associated binding occurs, nothing else can bind to it. The binding relationship is one-to-one. Nothing else can bind to the `PersistentVolume` once claimed. The following `get` command uses the short-form `pvc` instead of `persistentvolumeclaim`:

```
$ kubectl apply -f db-pvc.yaml
persistentvolumeclaim/db-pvc created
$ kubectl get pvc db-pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
db-pvc	Bound	db-pv	1Gi	RWO		111s

The `PersistentVolume` has not been mounted by a Pod yet. Therefore, inspecting the details of the object shows `<none>`. Using the `describe` command is a good way to verify that the `PersistentVolumeClaim` was mounted properly:

```
$ kubectl describe pvc db-pvc
...
Used By:          <none>
...
```

Binding by Volume Name

When creating a `PersistentVolumeClaim`, you can optionally specify the exact `PersistentVolume` you want to bind to using the `spec.volumeName` attribute. This creates a direct binding between the `PersistentVolumeClaim` and a specific `PersistentVolume`, bypassing the normal matching algorithm that considers storage size, access modes, and storage class.

This is useful in scenarios where:

- You have preprovisioned `PersistentVolumes` with specific characteristics.
- You need to ensure that a `PersistentVolumeClaim` binds to a particular `PersistentVolume` for compliance or data locality reasons.
- You want to reuse an existing `PersistentVolume` that contains important data.

[Example 16-4](#) shows an example of a PersistentVolumeClaim that explicitly binds to a PersistentVolume named `db-pv`.

Example 16-4. Binding a PersistentVolume to a PersistentVolumeClaim by name

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: specific-pvc
spec:
  volumeName: db-pv ❶
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

- ❶ Explicitly binds this PersistentVolumeClaim to the PersistentVolume named `db-pv`.

When using `volumeName`, ensure that:

- The specified PersistentVolume exists and is in `Available` status.
- The PersistentVolume’s capacity is greater than or equal to the PersistentVolumeClaim’s requested storage.
- The access modes are compatible.
- The storage classes match (or both are unset).

If these conditions aren’t met, the PersistentVolumeClaim will remain in `Pending` status.

Mounting PersistentVolumeClaims in a Pod

All that’s left is to mount the PersistentVolumeClaim in the Pod that wants to consume it. You already learned how to mount a volume in a Pod. The big difference here, shown in [Example 16-5](#), is using `spec.volumes[].persistentVolumeClaim` and providing the name of the PersistentVolumeClaim.

Example 16-5. A Pod referencing a PersistentVolumeClaim

```
apiVersion: v1
kind: Pod
metadata:
```

```

    name: app-consuming-pvc
spec:
  volumes:
    - name: app-storage
      persistentVolumeClaim: ❶
        claimName: db-pvc ❷
  containers:
    - image: alpine:3.22.2
      name: app
      command: ["/bin/sh"]
      args: ["-c", "while true; do sleep 60; done;"]
      volumeMounts:
        - mountPath: "/mnt/data"
          name: app-storage

```

❶ The volume type that selects a persistent volume claim by name

❷ The name of the persistent volume claim object we want to bind to

Let's assume we stored the configuration in the file *app-consuming-pvc.yaml*. After creating the Pod from the manifest, you should see the Pod transitioning into the Ready state. The `describe` command will provide additional information on the volume:

```

$ kubectl apply -f app-consuming-pvc.yaml
pod/app-consuming-pvc created
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
app-consuming-pvc                  1/1     Running   0           3s
$ kubectl describe pod app-consuming-pvc
...
Volumes:
  app-storage:
    Type:          PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)
    ClaimName:     db-pvc
    ReadOnly:      false
...

```

The PersistentVolumeClaim now also shows the Pod that mounted it:

```

$ kubectl describe pvc db-pvc
...
Used By:          app-consuming-pvc
...

```

You can now go ahead and open an interactive shell to the Pod.

Navigating to the mount path at */mnt/data* gives you access to the under-

lying PersistentVolume:

```
$ kubectl exec app-consuming-pvc -it -- /bin/sh
/ # cd /mnt/data
/mnt/data # ls -l
total 0
/mnt/data # touch test.db
/mnt/data # ls -l
total 0
-rw-r--r--    1 root    root          0 Sep 29 23:59 test.db
```

Storage Classes

A *StorageClass* is a Kubernetes primitive that defines a specific type or “class” of storage. One typical storage characteristic is the type (e.g., fast SSD storage versus remote cloud storage or the backup policy for storage). The storage class is used to provision a PersistentVolume dynamically based on its criteria.

In practice, this means that you do not have to create the PersistentVolume object yourself. The provisioner assigned to the storage class takes care of it. Most Kubernetes cloud providers come with a list of existing provisioners. minikube already creates a default storage class named `standard`, which you can query with the following command:

```
$ kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
standard (default)	k8s.io/minikube-hostpath	Delete	Immediate	false	108d

Creating Storage Classes

Storage classes can be created declaratively only with the help of a YAML manifest. At a minimum, you need to declare the provisioner. All other attributes are optional and use default values if not provided upon creation. Most provisioners let you set parameters specific to the storage type. [Example 16-6](#) defines a storage class on Google Compute Engine denoted by the provisioner `kubernetes.io/gce-pd`.

Example 16-6. Definition of a storage class

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
```

```

    name: fast
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
  replication-type: regional-pd

```

If you saved the YAML contents in the file *fast-sc.yaml*, then the following command will create the object. The storage class can be listed using the `get storageclass` command:

```

$ kubectl create -f fast-sc.yaml
storageclass.storage.k8s.io/fast created
$ kubectl get storageclass

```

NAME	PROVISIONER	RECLAIMPOLICY	\
VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE	
fast	kubernetes.io/gce-pd	Delete	\
Immediate	false	4s	
...			

Using Storage Classes

Provisioning a `PersistentVolume` dynamically requires assigning of the storage class when you create the `PersistentVolumeClaim`. [Example 16-7](#) shows the usage of the attribute `spec.storageClassName` for assigning the storage class named `standard`.

Example 16-7. Using a storage class in a `PersistentVolumeClaim`

```

kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: db-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 512Mi
  storageClassName: standard ❶

```

❶ Uses the storage class by its name to enable dynamic provisioning

The corresponding `PersistentVolume` object will be created only if the storage class can provision a fitting `PersistentVolume` using its provisioner. It's important to understand that Kubernetes does not render an error or warning message if this isn't the case.

The following command renders the created PersistentVolumeClaim and PersistentVolume. As you can see, the name of the dynamically provisioned PersistentVolume uses a hash to ensure a unique naming:

```
$ kubectl get pv,pvc
```

NAME	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	CAPACITY \ STORAGECLASS \
persistentvolume/pvc-b820b919-f7f7-4c74-9212-ef259d421734	RWO	Delete	Bound	default/db-pvc	512Mi \ standard \

NAME	STATUS	VOLUME
persistentvolumeclaim/db-pvc	Bound	pvc-b820b919-f7f7-4c74-9212-ef259d42
512Mi	RWO	standard

The steps for mounting the PersistentVolumeClaim from a Pod are the same as for static and dynamic provisioning. Refer to [“Mounting PersistentVolumeClaims in a Pod”](#) for more information.

Summary

PersistentVolumes store data beyond a Pod or cluster/node restart. Those objects are decoupled from the Pod’s lifecycle and are therefore represented by a Kubernetes primitive. The PersistentVolumeClaim abstracts the underlying implementation details of a PersistentVolume and acts as an intermediary between the Pod and PersistentVolume. A PersistentVolume can be provisioned statically by creating the object or dynamically with the help of a provisioner assigned to a storage class.

Exam Essentials

Internalize the mechanics of defining and consuming a PersistentVolume

Creating a PersistentVolume involves a couple of moving parts. Understand the configuration options for PersistentVolumes and PersistentVolumeClaims and how they play together. Try to emulate situations that prevent a successful binding of a PersistentVolumeClaim. Then fix the situation by taking counteractions. Internalize the short-form commands `pv` and `pvc` to save precious time during the exam.

Know the differences between static and dynamic provisioning of a PersistentVolume

A PersistentVolume must be created statically by creating the object from a YAML manifest. Alternatively, you can let Kubernetes provision a PersistentVolume dynamically without your direct involvement. For this to happen, assign a storage class to the PersistentVolumeClaim. The provisioner of the storage class takes care of creating the PersistentVolume object for you.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Create a PersistentVolume named `logs-pv` that maps to the `hostPath /var/logs`. The access mode should be `ReadWriteOnce` and `ReadOnlyMany`. Provision a storage capacity of 5 Gi. Ensure that the status of the PersistentVolume shows `Available`.
Create a PersistentVolumeClaim named `logs-pvc`. It uses `ReadWriteOnce` access. Request a capacity of 2 Gi. Ensure that the status of the PersistentVolume shows `Bound`.
Mount the PersistentVolumeClaim in a Pod running the image `nginx` at the mount path `/var/log/nginx`.
Open an interactive shell to the container and create a new file named `my-nginx.log` in `/var/log/nginx`. Exit out of the Pod.
Delete the Pod and re-create it with the same YAML manifest. Open an interactive shell to the Pod, navigate to the directory `/var/log/nginx`, and find the file you created before.
2. Navigate to the directory `app-a/ch16/dynamic-provisioning` of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Inspect the YAML manifest definition in the file `local-path-storage-0.0.31.yaml`.
Create the objects from the YAML manifest file.
Create a PersistentVolumeClaim named `db-pvc` in the namespace `persistence`. The access it uses is `ReadWriteOnce`. Request a capacity of 10 Mi. Use the storageclass name `local-path`.
Ensure that the status of the PersistentVolumeClaim object shows `Pending`. A PersistentVolume object should not have been provisioned yet.
Mount the PersistentVolumeClaim in a Pod named `app-consuming-pvc` in the namespace `persistence` at the mount path `/mnt/data`. The container should use the image `alpine:3.21.3`.
Wait until the Pod transitions into the `Running` status. Ensure that the PersistentVolume object has been created dynamically.
Open an interactive shell to the container, and create a new file named `test.db` in `/mnt/data`. Exit out of the Pod.

